EEM 364
Mikroişlemcileri

Assembly Dili Yapısı: Directives

Directives vs. Instruction Mnemonics

Data definition directives look a little like machine instruction mnemonics, but they are emphatically *not* machine instructions. One very common mistake made by beginners is looking for the binary opcode represented by a directive such as **DB** or **DW.** There is no binary opcode for **DW, DB,** and the other directives. Machine instructions, as the name implies, are instructions to the CPU itself. Directives, by contrast, are instructions to the *assembler*.

http://index-of.co.uk/Assembly/Assembly_Language_Step_by_Step_en.pdf

Assembly Dili Yapısı: Directives

```
.MODEL SMALL

Data SEGMENT

    veri db 1 , 2 , 3 , 4

Data ENDS


Code SEGMENT

Code ENDS
```

Assembly Dili Yapısı: Sembollerin Kullanımı

- **Semboller:** Verinin tutulduğu RAM bölgesinin başlangıcıdır.

- LEA ve OFFSET kullanımı ile adres değeri yazmaç içine çekilebilir

- veri db 5 DUP(12H) , veri db 5 DUP(12H,5AH)

veri

12
12
12

12
5A
12
5A
⋮

5 tane

Assembly Dili Yapısı: Sembollerin Kullanımı

- **BYTE PTR ,WORD PTR**

RAM bölgesinden ne kadarlık veri çekileceğini belirler

Assembly Dili Yapısı: Sembollerin Kullanımı

- **Semboller:** Verinin tutulduğu RAM bölgesinin başlangıcıdır.
- LEA ve OFFSET kullanımı ile adres değeri yazmaç içine çekilebilir
- veri db 5 DUP(12H) , veri db 5 DUP(12H,5AH)
- **BYTE PTR ,WORD PTR**

Assembly Dili Yapısı: Etiketler ve Döngüler

```
.MODEL SMALL

Data SEGMENT

x db 01H , 02H , 03H , 04H , 05H

Data ENDS

Code SEGMENT
mov ax , @Data
mov ds , ax

Code ENDS
```

→ Dışarıdan alınan verilerin toplanması (byte)

→ Sayaç Eklenmesi

→ Word sayıların toplanması

Assembly Dili Yapısı: Etiketler ve Döngüler

$a = 0$

$b = 1$

$t = 0$

$t \leftarrow b$

$b \leftarrow b + a$

$a \leftarrow t$

- Fibonacci Serisi

```
mov ax , 0
mov bl , 0

mov al , 0
mov ah , 1

mov bl , ah
add ah , al
mov al , bl

mov bl , ah
add ah , al
mov al , bl

mov bl , ah
add ah , al
mov al , bl

mov bl , ah
add ah , al
mov al , bl

mov bl , ah
add ah , al
mov al , bl
```

Assembly Dili Yapısı: Etiketler ve Döngüler

- **Etiket:** Yüksek seviye programlama dillerinde **goto** kullanımına benzer. Dallanma komutlarıyla çalışırlar.

Assembly Dili Yapısı: Etiketler ve Döngüler

Jump instructions that test single flag

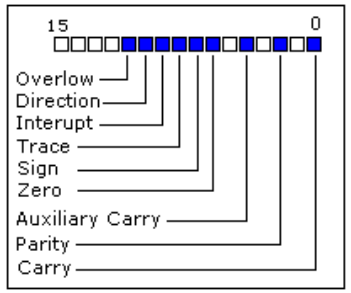
Instruction	Description	Condition	Opposite Instruction
JZ , JE	Jump if Zero (Equal).	ZF = 1	JNZ, JNE
JC , JB, JNAE	Jump if Carry (Below, Not Above Equal).	CF = 1	JNC, JNB, JAE
JS	Jump if Sign.	SF = 1	JNS
JO	Jump if Overflow.	OF = 1	JNO
JPE, JP	Jump if Parity Even.	PF = 1	JPO
JNZ , JNE	Jump if Not Zero (Not Equal).	ZF = 0	JZ, JE
JNC , JNB, JAE	Jump if Not Carry (Not Below, Above Equal).	CF = 0	JC, JB, JNAE
JNS	Jump if Not Sign.	SF = 0	JS
JNO	Jump if Not Overflow.	OF = 0	JO

http://www1.frm.utn.edu.ar/arquitectura/t86.pdf

8086 assembler tutorial for beginners (part 6)

Arithmetic and Logic Instructions

Most Arithmetic and Logic Instructions affect the processor status register (or **Flags**)



ADD, SUB,CMP, AND, TEST, OR, XOR

Assembly Dili Yapısı: Etiketler ve Döngüler

```
.MODEL SMALL

Data SEGMENT

x db 01H , 02H , 03H , 04H , 05H
y db 0

Data ENDS

Code SEGMENT

mov ax , @Data
mov ds , ax

mov ax , 0
mov di , 5
mov si , 0

TPL: add ah , x[si]
     inc si
     dec di
     jnz TPL

mov y , ah

hlt

Code ENDS
```

- Dizilerin toplanması (byte)

Etiketler ve Döngüler

```
.MODEL SMALL

Data SEGMENT

x db 01H , 02H , 03H , 04H , 05H
y db 0

Data ENDS

Code SEGMENT

mov ax , @Data
mov ds , ax

mov ax , 0
mov di , 5
mov si , 0

TPL: add ah , x[si]
     inc si
     dec di
     jnz TPL

mov y , ah

hlt

Code ENDS
```

- Dizilerin toplanması (word)

```
01 00 02 00 03 00 04 0
BB 10 07 BF DB B9 05 0
```

LAB Ödevi-1

- X db 1,2,3,4,5,6,7,8,9

Tek indisliler bir yerde çift indisliler bir yerde toplansın

İpucu: di ve si iki indis yazmacı kullanılır

12

test al, 01H

and al, 0FH

- Fibonacci serisi: Etiket kullanarak

0001 0010

0000 0001

0000 0000

0000 1111

3 3

0000 3

Assembly Dili Yapısı: Etiketler ve D

- Basit segment tanımlama ve
- EQU direktifi

```
.MODEL SMALL

.DATA

x db 1,2,3,4,5

toplam db ?

.CODE


mov ax , @data
mov ds , ax


mov si , 0
mov cx , 5
mov al , 0

TPL: add al , x[si]
     inc si
     dec cx
     jnz TPL

mov toplam , al


hlt
```

Assembly Dili Yapısı: Etiketler ve Döngüler

- Basit segment tanımlama ve
- EQU direktifi

```
01 .MODEL SMALL
02
03 .DATA
04
05
06 x  db 1, 2, 15, 33, 66, 5, 3, 8, 9, 25, 34, 22
07 lx equ $-x
08 ts db 25
09
10
11 U db ?
12 V db ?
13
14 .CODE
15
16 mov ax , @data
17 mov ds , ax
18
```

Assembly Dili Yapısı: Etiketler ve Döngüler

- **JNZ/JZ**
- **INC**
- **DEC**
- **OFFSET**
- **EQU**
- **LEA**

- WORD sayı çekerken indisi 2 arttırmak !!
- EQU ile tanımlanan sembol hem 8 hem de 16 bit içine atanabilir. Sabit Sayıdır !!
- $-veri sayesinde dizilerin uzunlukları bulunur

Koşullar ve Döngüler

- JNZ/JZ
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL

```
CMP register, register;
CMP AX, BX
CMP register, memory;
CMP AX, SONUC
CMP register,
  immediate;
CMP AX,5
```

```
CMP memory, register;
CMP SONUC, AX
CMP memory, immediate;
CMP SONUC,5
```

https://web.karabuk.edu.tr/emelkocak/indir/MTM305/Kontrol_Komutlar%C4%B1.pdf

Assembly Dili Yap

Koşul Kurma

Belli bir sayıdan küçük olanları toplamak

```
.MODEL SMALL

.DATA


x  db 1, 2, 15, 33, 66, 5, 3, 8, 9, 25, 34, 22
lx equ $-x
ts db 10

TP db 0

.CODE

mov ax , @data
mov ds , ax



mov si , 0
mov cx , lx
mov al , 0
mov ah , 0

KAR: mov bl , x[si]
     cmp bl , ts
     jl TPL
     inc si
     dec cx
     jnz KAR



SON:
mov TP , al



hlt

TPL:  add al , bl
      inc si
      dec cx
      jz  SON
      jmp KAR
```

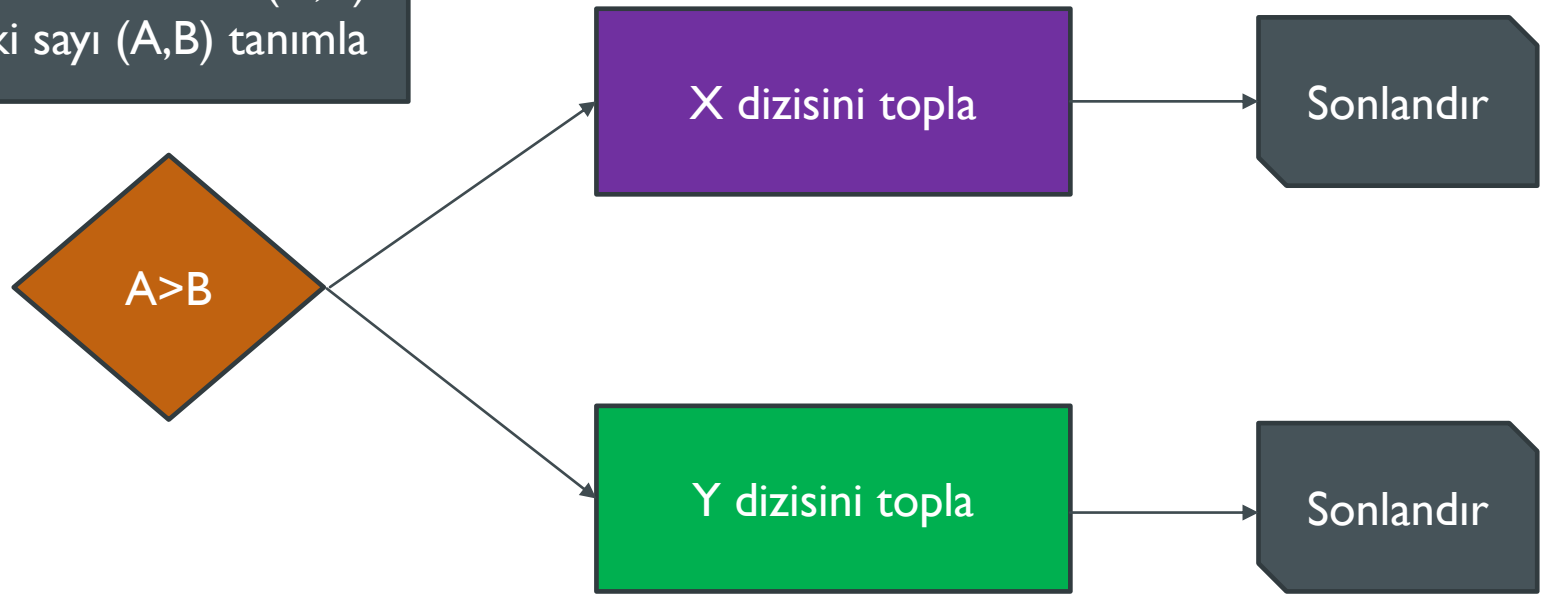
Koşullar

Dizi içindeki elemanlar belli bir sayıdan büyükse U'da değilse V'de toplansın

```
.MODEL SMALL

.DATA


x  db 1, 2, 15, 33, 66, 5, 3, 8, 9, 25, 34, 22
lx equ $-x
ts db 25


U db ?
V db ?

.CODE

mov ax , @data
mov ds , ax

mov si , 0
mov cx , lx
mov al , 0
mov ah , 0


KAR: mov bl , x[si]
     cmp bl , ts
     jl VL
     jg UL


SON:
mov V , al
mov U , ah


hlt

VL:  add al , bl
     inc si
     dec cx
     jz  SON
     jmp KAR

UL:  add ah , bl
     inc si
     dec cx
     jz  SON
     jmp KAR
```

Koşullar ve Döngüler

İki dizi tanımla (X,Y)
İki sayı (A,B) tanımla



Koşullar ve Döngüler

```
.MODEL SMALL

.DATA

x   db    1,2,3,4,5
lx  equ $-x

y   db    2,4,6,8,10
ly  equ $-y

A   db 9
B   db 6

TX  db ?
TY  db ?

.CODE

mov ax , @data
mov ds , ax

mov cl , lx
mov ch , ly

mov si , 0
mov bx , 0

mov ah , A
mov al , B

cmp ah,al

jg  top_x
jl  top_y


hlt
```

```
hlt

top_y:
      add bh , y[si]
      inc si
      dec ch
      jnz top_y

mov TY,bh

hlt


top_x:
      add bl , x[si]
      inc si
      dec cl
      jnz top_x

mov TX,bl

hlt
```

Koşullar ve Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST

Bit Tabanlı AND İşlemi

```
.MODEL SMALL

.DATA

.CODE

mov ax , @data
mov ds , ax


mov al , 5
mov bl , 8

test al , 01H
test bl , 01H


hlt
```

Koşullar ve Döngüler

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST

Bit Tabanlı AND İşlemi

```
.MODEL SMALL

.DATA

.CODE

mov ax , @data
mov ds , ax


mov al , 5
mov bl , 8

test al , 01H
test bl , 01H


hlt
```

Sayıları çift ve tek olma durumuna göre toplama

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST

```
.DATA

x  db 1, 2, 15, 33, 66, 5, 3, 8, 9, 25, 34, 22
lx equ $-x

U db ?
V db ?

.CODE

mov ax , @data
mov ds , ax



mov si , 0
mov cx , lx
mov al , 0
mov ah , 0


KAR: mov bl , x[si]
     test bl , 01H
     jz   CFT
     jnz  TEK


SON:
mov V , al
mov U , ah


hlt

CFT:  add al , bl
      inc si
      dec cx
      jz  SON
      jmp KAR

TEK:  add ah , bl
      inc si
      dec cx
      jz  SON
      jmp KAR
```

Koşullar ve Döngüler ve Prosedürler !!

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST
- PROC

```
.MODEL SMALL

.DATA
.CODE


main proc far
        mov ax , @data
        mov ds , ax

        call F1

        mov ax , bx


        hlt
main endp


F1 PROC
       mov bx , 1233H
       ret
F1 ENDP
```

Koşullar ve Döngüler ve Prosedürler !!

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST
- PROC
- LOOP

```
.MODEL SMALL

.DATA
.CODE


main proc far
        mov ax , @data
        mov ds , ax

        call F1

        mov ax , bx


        hlt
main endp


F1 PROC
       mov bx , 1233H
       ret
F1 ENDP
```

Loop Komutu İle Toplama

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA
- CMP
- JE , JG , JL
- TEST
- PROC
- LOOP

```
.MODEL SMALL


.DATA
    x  db 1, 2, 15, 33, 66, 5, 3, 8, 9, 25, 34, 22
    lx equ $-x
    T db ?

.CODE


main proc far

     mov ax , @data
     mov ds , ax

     call F1

     mov T , al

     hlt

main endp


F1 proc

    mov cx , lx
    mov al , 0

    lea si , x

    L:
      add al , [si]
      inc si
      loop L

    ret

F1 endp
```

LAB ÖDEVİ-1

x db 1,2,4,5,7,1 y db 3,1,5,2,2,4 iki tane dizi olsun |x(i) - y(i)| değerlerini bulunuz. Bunları M sembolüne atayınız.

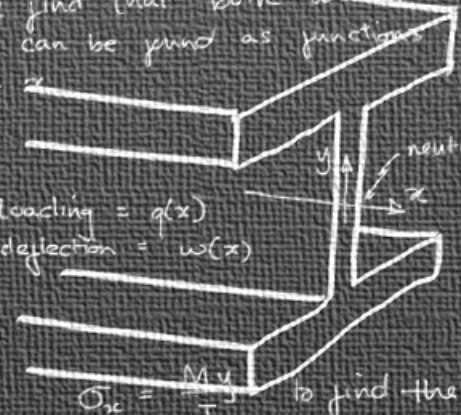
İpucu: Mutlak değer işlemi asm'de bulunmamaktadır. Sayıları büyük ve küçük olarak karşılaştırıp her zaman büyük olandan küçük olanı çıkaracak şekilde bir koşul yazmanız gerekecektir.

LAB ÖDEVİ-2

x db 1,2,4,5,7,1

Sayılarını toplayan prosedürü yazınız. Sonucu T sembolüne yazdırınız.

given a beam of constant EI
we find that both w and
q can be found as functions
of x



loading = $q(x)$
deflection = $w(x)$

$\sigma_x = \frac{My}{I}$ to find the tension force at some location y

for a static system

$$\frac{\partial^2}{\partial x^2}\left(EI\frac{\partial^2 w}{\partial x^2}\right) = q$$

for $\frac{\partial EI(x)}{\partial x} = 0$

where $\frac{\partial^4 w}{\partial x^4} EI = q(x)$

E = Young's Modulus

I = Second moment of area

http://bsnscb.com/data/out/124/39800806-maths-wallpapers.jpg